Day 98 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 1 reference

Watch: Getting Started With Hugging Face in 15 Minutes (AssemblyAI) | What is BERT? AI Explained Simply (Coursesteach)

`day98_pretrained_transformers.py`, segment by segment, in the order `main()` uses things. This is the first script that uses PyTorch and the Hugging Face libraries; the from-scratch parts (bootstrap, calibration, the bag-of-words model) are NumPy as before. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: z = logits (raw scores) · p = probabilities · θ_w = one word's weight · n_b = answers in bin b · ECE = expected calibration error

Segment 0 — Setup

Imports and settings

CODE

```
import torch
from datasets import disable_progress_bars, load_dataset
from transformers.utils import logging as hf_logging
from huggingface_hub.utils import logging as hub_logging
from transformers import AutoModelForSequenceClassification, AutoTokenizer, pipeline
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
MODEL = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
THREADS = 4
BATCH = 32
N_BOOT = 2000
TRAIN_SIZES = (1000, 10000, 67349)
torch.set_num_threads(THREADS)
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
disable_progress_bars()
```

- `from transformers import AutoModelForSequenceClassification, AutoTokenizer, pipeline` — the 'Auto' classes read the model's saved config and build the matching class (here DistilBERT).
- `torch.set_num_threads(THREADS)` — how many CPU threads PyTorch may use for its matrix work.
- `hf_logging.set_verbosity_error()` — only errors from the library reach the output, so the run output is the lesson's own.

Segment 1 — The library

load and sst2: the model and the data

CODE

```
def load():
    tok = AutoTokenizer.from_pretrained(MODEL)
    model = AutoModelForSequenceClassification.from_pretrained(MODEL).eval()
    return tok, model
def sst2(split):
    d = load_dataset("nyu-mll/glue", "sst2", split=split)
    return list(d["sentence"]), np.array(d["label"])
```

- `AutoTokenizer.from_pretrained(MODEL)` — downloads the tokenizer files on first use, then reads them from the local cache.
- `.eval()` — switches dropout off: inference must be deterministic.
- `load_dataset("nyu-mll/glue", "sst2", split=split)` — one split of a dataset from the Hugging Face hub, as a table of columns.

predict_proba: batched inference

MATH

$$p_k = e^{z_k} / \Sigma_j e^{z_j}$$

CODE

```
def predict_proba(tok, model, texts, bs=BATCH):
    out = []
    with torch.no_grad():
        for i in range(0, len(texts), bs):
            enc = tok(texts[i:i + bs], padding=True, truncation=True, return_tensors="pt")
            out.append(torch.softmax(model(**enc).logits, dim=-1).numpy())
    return np.concatenate(out)
```

- `with torch.no_grad():` — a context manager: no gradient bookkeeping inside it, which saves memory and time.
- `tok(..., padding=True, truncation=True, return_tensors="pt")` — pads to the batch's longest input, cuts anything over 512 tokens, returns PyTorch tensors.
- `model(**enc).logits` — **enc passes input_ids and attention_mask as keyword arguments; .logits are the two raw scores.
- `.numpy()` — a PyTorch tensor back to a NumPy array, for the from-scratch code.

Segment 2 — From-scratch measures

bootstrap_ci and calibration

MATH

$$ECE = \Sigma_b \, (n_b / N) \, | \, conf_b - acc_b \, |$$

CODE

```
def bootstrap_ci(correct, n_boot=N_BOOT, seed=0):
    rng = np.random.RandomState(seed)
    idx = rng.randint(0, len(correct), (n_boot, len(correct)))
    accs = correct[idx].mean(axis=1)
    return np.percentile(accs, 2.5), np.percentile(accs, 97.5)
def calibration(conf, correct, n_bins=10):
    edges = np.linspace(0.5, 1.0, n_bins + 1)
    rows, ece = [], 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi) if lo > 0.5 else (conf >= lo) & (conf <= hi)
        if m.sum() == 0:
            continue
        rows.append((lo, hi, m.sum(), conf[m].mean(), correct[m].mean()))
        ece += m.mean() * abs(conf[m].mean() - correct[m].mean())
    return rows, ece
```

- `rng.randint(0, len(correct), (n_boot, len(correct)))` — a matrix of indices: each row is one resample, drawn with replacement.
- `correct[idx].mean(axis=1)` — fancy indexing builds all 2,000 resamples at once; one accuracy per row.
- `(conf > lo) & (conf <= hi)` — element-wise AND of two boolean arrays: the answers in this bin.

BagOfWords: logistic regression without a matrix

MATH

$$s = b + \Sigma_{w \text{ in } x} \, \theta_w, \quad \partial L / \partial \theta_w = \Sigma_{x \text{ containing } w} \, (p_x - y_x)/N + \lambda \theta_w$$

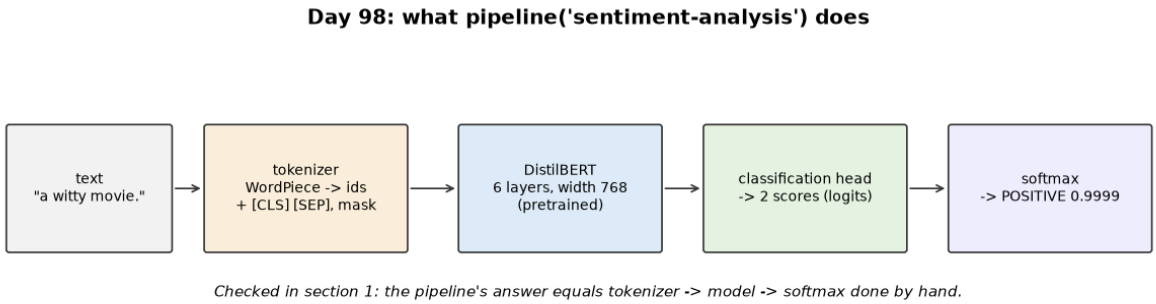
CODE

```
def encode(self, texts):
    ids, rows = [], []
    for r, t in enumerate(texts):
        for w in set(words(t)):
            if w in self.vocab:
                ids.append(self.vocab[w]); rows.append(r)
    return np.array(ids, dtype=np.int64), np.array(rows, dtype=np.int64)
def scores(self, ids, rows, n):
    return self.b + np.bincount(rows, weights=self.w[ids], minlength=n)
def fit(self, texts, y, steps=300, lr=0.05, l2=1e-4):
    ids, rows = self.encode(texts); n = len(texts)
    self.w, self.b = np.zeros(len(self.vocab)), 0.0
    m, v = np.zeros(len(self.vocab) + 1), np.zeros(len(self.vocab) + 1)
    for t in range(1, steps + 1):
        p = 1 / (1 + np.exp(-self.scores(ids, rows, n)))
        g_s = (p - y) / n
        g = np.r_[np.bincount(ids, weights=g_s[rows], minlength=len(self.vocab)) + l2 * self.w,
            g_s.sum()]
        m = 0.9 * m + 0.1 * g; v = 0.999 * v + 0.001 * g ** 2
        step = lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
        self.w -= step[:-1]; self.b -= step[-1]
```

- `np.bincount(rows, weights=self.w[ids], minlength=n)` — sums each sentence's word weights in one call: a sparse matrix product without the matrix.

- `g_s = (p - y) / n` — the gradient of the mean log-loss with respect to each sentence's score.
- `np.r_[..., g_s.sum()]` — joins the word gradients and the bias gradient into one vector for Adam.

SYSTEM DIAGRAM



What pipeline('sentiment-analysis') does.

Segment 3 — The experiments

demo_library: the pipeline against the manual steps

CODE

```
pipe = pipeline("sentiment-analysis", model=model, tokenizer=tok)
res = pipe(texts)
enc = tok(texts, padding=True, return_tensors="pt")
with torch.no_grad():
    logits = model(**enc).logits
probs = torch.softmax(logits, dim=-1).numpy()
```

- `pipeline("sentiment-analysis", model=model, tokenizer=tok)` — reuses the already-loaded objects instead of loading them again.
- `res = pipe(texts)` — a list of dictionaries, one per text: {'label': ..., 'score': ...}.
- `torch.softmax(logits, dim=-1)` — softmax over the last axis: the two classes.

demo_tokenizer: pieces, ids and the mask

CODE

```
for s in ("The movie wasn't unwatchable, just forgettable.", "Spielberg's best since Jaws"):
    print(f"  {s!r} -> {tok.tokenize(s)}")
enc = tok(X[:3], padding=True)
print("\nthree validation sentences padded into one batch (ids and attention mask of the
    shortest):")
k = int(np.argmin([sum(m) for m in enc["attention_mask"]]))
print(f"  ids  {enc['input_ids'][k]}\n  mask {enc['attention_mask'][k]}")
print(f"  [CLS] = {tok.cls_token_id}, [SEP] = {tok.sep_token_id}, [PAD] = {tok.pad_token_id}")
```

- `tok.tokenize(s)` — the WordPiece pieces as strings, without special tokens or ids.
- `enc["input_ids"], enc["attention_mask"]` — without return_tensors the tokenizer returns plain Python lists.
- `tok.cls_token_id` — the id of a special token, read from the tokenizer instead of hard-coded.

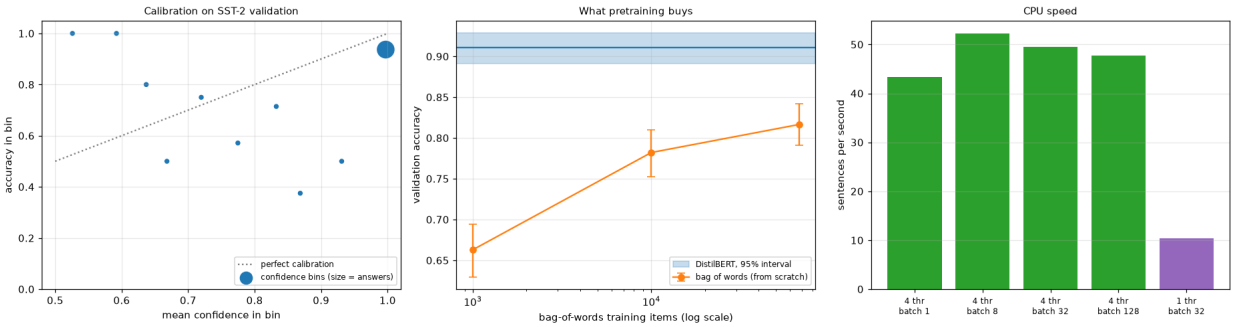
demo_speed: threads and batch sizes

CODE

```
for th, bs in ((THREADS, 1), (THREADS, 8), (THREADS, 32), (THREADS, 128), (1, 32)):
    torch.set_num_threads(th)
    ts = []
    for _ in range(3):
        t0 = time.perf_counter(); predict_proba(tok, model, texts, bs=bs);
            ts.append(time.perf_counter() - t0)
    res[(th, bs)] = len(texts) / np.median(ts)
    print(f"  {th} thread(s), batch {bs:>3}: {res[(th, bs)]:6.1f} sentences per second")
torch.set_num_threads(THREADS)
```

- `torch.set_num_threads(th)` — changes PyTorch's thread count for the timings that follow.
- `np.median(ts)` — the middle of three timings, less affected by one slow run than the mean.

REAL GRAPH



Real graph from the script: calibration, the bag-of-words baseline against DistilBERT, and CPU speed.

Quick Syntax Glossary — new or reinforced constructs

- `Auto*.from_pretrained(name)` — download (once) and build a tokenizer or model from the hub by name.
- `with torch.no_grad():` — run code without recording gradients.
- `model(**enc)` — unpack a dictionary of tensors as keyword arguments.
- `tensor.numpy()` — a PyTorch tensor as a NumPy array (CPU tensors without gradients).
- `np.bincount(idx, weights=w, minlength=n)` — sum the weights that fall into each index: a fast group-by sum.
- `np.percentile(a, q)` — the value below which q% of a falls.

End of Day 98 syntax reference. For the experiments and what they showed, see the Day 98 complete guide.